Chandigarh University

UNIVERSITY INSTITUTE OF COMPUTING

CASE STUDY

STUDENT RECORD MANAGEMENT SYSTEM

SUBMITTED BY.

Student Name: Gaurav Soni

UID : 25BCD10090

Programme: BCA Data Science

Semester: 3rd

SUBMITTED To

Faculty name: Mr. Jitendra Kumar Sir

Subject: Object Oriented Programming

Subject code: 25CAH-201

Abstract

This detailed case study documents the analysis, design, implementation, and testing approach for a beginner-level Student Record Management System developed in C++ using Object-Oriented Programming. The application manages student roll numbers, names, and marks during a single execution. It supports adding, displaying, searching, updating, and deleting records. The report explains the requirements, system modules, algorithms, OOP concepts, complexity, validation, limitations, and future enhancements. Since records are held in memory, they are not retained after the program closes.

1. Introduction

The Student Record Management System is a beginner-level console application developed in C++ using Object-Oriented Programming (OOP). The program manages three basic details for each student: roll number, name, and marks. It provides a menu through which the user can add, display, search, update, and delete student records.

Small programming projects help students connect theoretical concepts with practical problem-solving. This project uses a familiar educational scenario to demonstrate how a class can represent a real-world entity and how member functions can operate on its data.

This version is intentionally limited in scope. It stores records in memory using a vector, so information is available only while the application is running. Permanent storage, database connectivity, and a graphical interface are possible future improvements rather than existing features.

2. Background of the Study

Student details are frequently used in educational settings. In a manual process, a person may need to scan a notebook or spreadsheet to find a record, cross out an old value, or rewrite a corrected entry. These steps can become repetitive and may lead to inconsistencies when the same details are copied in multiple places.

A small computer program can centralize basic operations in one place. The purpose of this project is not to replace a full college information system; it is to demonstrate how introductory C++ techniques can make a simple set of records easier to manage.

The case study also provides a practical context for OOP. A student has attributes such as a roll number, name, and marks, and operations such as displaying or updating details. These attributes and operations can be grouped inside a class.

3. Problem Statement

Manual handling of a small collection of student records can make it inconvenient to find a particular student, change marks or names, and remove an obsolete entry. The project addresses this problem by providing a menu-driven program that performs common record operations and checks for duplicate roll numbers during insertion.

The system focuses on basic correctness and clarity rather than advanced features. It demonstrates how a collection of objects can be searched and modified using simple functions and control structures.

4. Aim and Objectives

Aim: To design and implement a simple Student Record Management System using C++ and Object-Oriented Programming.

Specific objectives:

• Represent student information through a dedicated class.

• Keep the class data members private and expose selected operations through public functions.

• Store multiple student objects dynamically during execution.

• Add a new record only when its roll number is not already present.

• Display all records in a readable table.

• Search for a record by roll number.

• Update the name and marks of an existing student.

• Delete an existing record by roll number.

• Validate marks so that they fall between 0 and 100.

• Demonstrate classes, objects, encapsulation, constructors, vectors, loops, and selection statements.

5. Scope of the Project

In scope:

• A text-based menu for interacting with the program.

• Student records consisting of roll number, name, and marks.

• Add, display, search, update, and delete operations.

• Duplicate roll-number checking when adding a record.

• Basic marks validation.

• Execution on common operating systems with a suitable C++ compiler.

Out of scope:

• Permanent saving to a file or database.

• Multiple user accounts or permissions.

• Network access and simultaneous users.

• Attendance, fee, timetable, or course management.

• A graphical user interface or web interface.

These boundaries keep the project achievable for a beginner and make it easier to explain during a practical demonstration or viva.

6. Feasibility Study

Technical feasibility: The project uses standard C++ features and standard library headers. It can be compiled with a C++17-compatible compiler such as GCC or Clang. No external service, database, or paid software is required.

Economic feasibility: The project can be developed using freely available tools such as GCC and Visual Studio Code. It does not require specialized hardware beyond a typical computer.

Operational feasibility: The menu presents numbered options, so a user can select an operation without learning complex commands. The system is small enough to understand and demonstrate in a classroom.

Schedule feasibility: Because the application has a limited number of fields and operations, it is suitable for a short academic mini-project. Additional features can be added later if time permits.

7. Requirements Analysis

Functional requirements:

FR1 — The program shall display a menu of operations.

FR2 — The program shall add a student when valid details are entered and the roll number is unique.

FR3 — The program shall display all stored student records.

FR4 — The program shall search by roll number.

FR5 — The program shall update the name and marks of an existing student.

FR6 — The program shall delete a record when the roll number exists.

FR7 — The program shall report when a record cannot be found.

FR8 — The program shall reject duplicate roll numbers during insertion.

FR9 — The program shall accept marks only within the range 0–100.

FR10 — The program shall terminate when the user selects Exit.

Non-functional requirements:

• Usability: menu choices and messages should be clear.

• Portability: source code should use standard C++ features.

• Maintainability: operations should be separated into a class and helper function.

• Reliability: common invalid inputs should not crash the program under normal testing.

• Performance: operations should be responsive for a small number of records.

8. Hardware and Software Requirements

Hardware:

• Desktop computer or laptop.

• Keyboard and monitor.

• At least 2 GB RAM recommended for the development environment.

Software:

• Windows, macOS, or Linux.

• A C++17-compatible compiler, such as GCC/G++ or Clang.

• A code editor or IDE, such as VS Code or Code::Blocks.

• Terminal or command prompt.

The project uses standard headers including iostream for input/output, vector for a dynamic collection, string for names, limits for clearing invalid input, and iomanip for formatted table output.

9. System Architecture and Module Design

The program follows a simple single-process console architecture. The user interacts with the menu, the main function selects an operation, and the vector holds the current collection of Student objects. The Student class defines the data and operations associated with one student.

Module 1 — Student class: stores roll number, name, and marks. Its constructor initializes the fields; getRollNo() returns the roll number; display() prints a record; update() changes the name and marks.

Module 2 — Search helper: findStudent() loops through the vector and compares each student's roll number with the requested roll number. It returns the matching index or -1 when no match is found.

Module 3 — Add: reads the roll number, checks whether it already exists, reads the name and marks, validates marks, and inserts a new Student object.

Module 4 — Display: checks whether the vector is empty. If it contains records, the program prints a heading and displays each student.

Module 5 — Search: uses findStudent() to locate a record and displays it when found.

Module 6 — Update: finds the selected record and calls update() with the new name and marks.

Module 7 — Delete: finds the record and removes it from the vector using erase().

Module 8 — Menu and exit: a do-while loop repeats the menu, and a switch statement dispatches the selected operation.

10. Data Design

The current program uses an in-memory vector rather than a database table. Each element is a Student object with the following logical fields:

Field: rollNo — type: integer — purpose: student identifier used for searching.

Field: name — type: string — purpose: student's name; spaces are supported.

Field: marks — type: floating-point number — purpose: marks in the range 0 to 100.

Validation rules:

• A roll number must be numeric.

• A roll number must not already exist when adding a record.

• A name is read as a line so it may contain spaces.

• Marks must be numeric and between 0 and 100.

Because no file or database is used, the data is not persistent. Closing the program clears the vector.

11. Algorithm and Pseudocode

Main algorithm:

1. Start.

2. Create an empty vector of Student objects.

3. Display the menu.

4. Read the user's choice.

5. Execute the matching operation.

6. Display a result or error message.

7. Return to the menu unless the choice is Exit.

8. Stop.

Pseudocode:

START

students ← empty collection

REPEAT

DISPLAY menu

READ choice

CASE choice OF

Add: read roll; reject if duplicate; read name and valid marks; add record

Display: show all records or empty message

Search: find by roll; show record or not-found message

Update: find by roll; read new name and valid marks; update record

Delete: find by roll; remove record or show not-found message

Exit: display goodbye message

Other: display invalid-choice message

END CASE

UNTIL choice = Exit

STOP

Flow description: Start → Display Menu → Read Choice → Execute Selected Operation → Show Result → Return to Menu → Exit → Stop. Each operation returns to the main menu after completion.

12. Object-Oriented Programming Concepts

Class: A class is a user-defined type that groups data and functions. The Student class models one student's record.

Object: An object is an instance of a class. When a new record is added, the program constructs a Student object and stores it in the vector.

Encapsulation: rollNo, name, and marks are private. This prevents unrelated code from changing the fields directly. Public methods provide a controlled interface.

Constructor: Student(int roll, const string& studentName, float studentMarks) initializes an object when it is created. The constructor uses an initializer list.

Abstraction: The menu lets users choose an operation without requiring them to understand the internal search or vector operations.

Member functions: getRollNo(), display(), and update() define behaviours associated with a Student object.

STL vector: vector<Student> is a standard library container that can grow or shrink as records are added and deleted. It is not an OOP pillar, but it is useful in implementing the collection.

Control structures: a do-while loop repeats the menu, a switch statement selects an operation, and for loops traverse the collection.

Why inheritance and polymorphism are not used: this small problem does not require different types of students or interchangeable behaviours. Adding those concepts only to claim their use would make the project more complex without improving its purpose.

13. Implementation Walkthrough

Header files: the program includes standard headers for console input/output, vector storage, strings, numeric limits, and formatted output.

Student class: private fields store the student's data. The constructor initializes them. getRollNo() supports searching; display() prints formatted details; update() changes the name and marks.

findStudent(): the helper examines each object and compares its roll number. It returns the index if found, otherwise -1. The calling operation uses this result to decide what to do.

Add operation: the program reads a roll number and checks for duplicates before asking for the remaining details. Marks are checked against the allowed range. A valid record is added with emplace_back().

Display operation: the program checks whether the vector is empty. If not, it prints column headings and calls display() for each object.

Search operation: the user enters a roll number. If findStudent() returns a valid index, the corresponding record is shown. Otherwise, a not-found message appears.

Update operation: the selected record is found first. If it exists, new values are read and passed to update(). The roll number remains unchanged.

Delete operation: the program searches for the roll number and uses vector::erase() to remove the matching element.

Input handling: the program clears the input stream when a non-numeric menu choice or roll number is entered. Marks are checked for numeric input and the 0–100 range. This is basic input validation appropriate for a beginner project.

14. Compilation and Execution

From the project root, compile the program with:

g++ -std=c++17 -Wall -Wextra -pedantic src/main.cpp -o student_records

On macOS or Linux, run:

./student_records

On Windows with MinGW, run:

student_records.exe

If compilation fails, check that a C++ compiler is installed and that the terminal is in the project root. A screenshot should be captured only after running the program locally.

15. Sample Interaction

Example: adding one student.

===== STUDENT RECORD MANAGEMENT SYSTEM =====

1. Add Student

2. Display All Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 1

Enter Roll Number: 101

Enter Name: Rahul Sharma

Enter Marks (0-100): 85

Student added successfully!

Example: displaying records.

Roll No Name Marks

---

101 Rahul Sharma 85.00

The sample is illustrative. For the final report, run the actual program and capture screenshots of add, display, search, update, and delete operations.

16. Testing Strategy and Test Cases

Testing checks whether each menu operation produces the expected result. Tests should include both valid inputs and common error cases.

TC01 — Add valid record: enter roll 101, name Rahul Sharma, marks 85. Expected: record added.

TC02 — Duplicate roll number: add roll 101 again. Expected: duplicate rejected.

TC03 — Display records: select option 2 after adding a record. Expected: the record is displayed.

TC04 — Search existing roll: search for 101. Expected: record found and displayed.

TC05 — Search missing roll: search for 999. Expected: not-found message.

TC06 — Update existing record: change roll 101 to a new name and marks. Expected: update confirmation and new values on display.

TC07 — Delete existing record: delete roll 101. Expected: deletion confirmation; subsequent search does not find it.

TC08 — Invalid marks: enter -5 or 105. Expected: invalid marks message and no record update/addition.

TC09 — Display empty collection: start the program and choose Display before adding records. Expected: no records message.

TC10 — Exit: select option 6. Expected: goodbye message and program termination.

TC11 — Invalid menu choice: enter a number outside 1–6. Expected: invalid-choice message.

TC12 — Restart persistence check: add a record, exit, and restart. Expected: the record is absent because the current version uses in-memory storage.

Testing note: The source was compiled successfully in the package-building environment and the exit-menu smoke test passed. Other scenarios should be run locally and documented with genuine outputs before submission.

17. Complexity Discussion

Let n be the number of student records currently stored.

Search: findStudent() checks records one by one, so its time complexity is O(n) in the worst case.

Display: the program visits each record once, so displaying all records is O(n).

Add: duplicate checking takes O(n); appending to a vector is amortized O(1), so the full add operation is O(n) because of the duplicate check.

Update: locating the record takes O(n); changing its fields after finding it is O(1).

Delete: locating the record takes O(n), and removing an element from the vector may shift later elements, which is O(n) in the worst case.

Space: storing n student records requires O(n) space. These costs are appropriate for a small learning project.

18. Data Integrity and Responsible Use

The program checks for duplicate roll numbers during addition and validates marks between 0 and 100. These checks reduce simple data-entry errors.

The application has no authentication, encryption, access control, audit history, or permanent storage. It should therefore be treated as a learning demonstration and not used to store confidential real student information.

If the project is extended, additional validation should cover blank names and acceptable roll-number rules. File or database storage should be designed carefully, and personal information should be protected according to applicable institutional rules.

19. Advantages

• Simple menu-based interaction.

• Clear example of class and object design.

• Private data members demonstrate encapsulation.

• Supports core create, read, update, and delete operations.

• Uses standard C++ libraries and needs no database.

• Small enough to compile, test, and explain in a viva.

• Can be extended incrementally.

20. Limitations

• Data is lost when the program exits.

• The application is designed for one local user at a time.

• No GUI, database, authentication, or access permissions are included.

• Search is linear and is not optimized for very large collections.

• Input validation is basic and not intended for production use.

• There is no audit trail for changes or deletions.

21. Future Enhancements

1. File handling: save records to a text or CSV file and reload them at startup.

2. Database integration: use a database to support persistent and structured storage.

3. Authentication: add login and user roles if multiple people use the application.

4. GUI: build a graphical interface for easier navigation.

5. Additional fields: add course, semester, email, or attendance if required by the project scope.

6. Reporting: calculate grades, averages, or student summaries.

7. Better validation: check empty names, roll-number format, and malformed input consistently.

8. Automated tests: create repeatable tests for all operations and error cases.

22. Conclusion

The Student Record Management System demonstrates how a familiar record-management task can be implemented using C++ and introductory OOP. It provides a menu-driven way to add, display, search, update, and delete student records.

The project illustrates the use of a class, objects, encapsulation, a constructor, member functions, vectors, loops, and selection statements. It also highlights the importance of validating input and testing expected and unexpected cases.

The main limitation is that the records exist only during program execution. Permanent storage, stronger validation, and access controls would be necessary for a more complete system. As a BCA mini-project, the current version provides a manageable foundation for learning and future extension.

23. References

1. Bjarne Stroustrup, Programming: Principles and Practice Using C++.

2. C++ reference documentation: https://en.cppreference.com/.

3. C++ standard library documentation for vector and string.

4. Course notes for Object-Oriented Programming Using C++.

5. Source code, README, and test plan included in this project package.

Appendix B. Sample Menu

===== STUDENT RECORD MANAGEMENT SYSTEM =====

1. Add Student

2. Display All Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 1

Enter Roll Number: 101

Enter Name: Rahul Sharma

Enter Marks (0-100): 85

Student added successfully!